# MARLET: Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring

Anonymous Authors

Abstract— Large language models can generate fluent tutoring responses, but they need grounding that respects both instructional evidence and the learner's current state. Classical retrieval-augmented generation (RAG) grounds generation through a fixed retrieve-then-read controller: the query is matched against a corpus, and the generator reads the returned passages. That design is often insufficient for tutoring, where the same surface question can require different evidence, scaffolding, or rubric constraints for different learners. We propose MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring), which integrates supervisory routing, planning, learner-state diagnosis, rubric analysis, conditional retrieval, tutoring, and bounded critique around a fixed tutor. MARLET builds a tutoring brief from the sample, inferred learning state, and answer criteria, then invokes retrieval only when the route indicates that external evidence is needed. Under matched backbones, corpus, reranker, and response budget on TutorEval and EduBench, MARLET improves fixed-tutor quality metrics while using fewer tokens; on EduBench it also reduces latency and backbone-call count. These results support the central claim: educational grounding should be controlled by pedagogical state and task regime, not by the surface query alone.

## I. Introduction

Large language models (LLMs) are increasingly used as educational tutors because they can generate explanations, diagnose errors, and provide feedback [1], [2]. However, an LLM tutor cannot rely only on parametric knowledge. Knowledge-grounded dialogue has long shown the value of external evidence [3]; educational responses similarly need grounding in textbook chapters, course notes, rubrics, worked examples, or learner profiles to avoid fluent but misaligned guidance.

Classical retrieval-augmented generation (RAG) has therefore become the default grounding mechanism [4]. In classical RAG, the query is embedded, the top-$k$ most similar passages are retrieved, and the LLM generates a response conditioned on those passages. Thus, relevance depends heavily on retrieval quality and query representation [5].

Educational tutoring is more complex than factual question answering. A student's question is often only a partial signal: the appropriate response may depend on level, goals, weak points, recent confusion, dialogue, and likely misconceptions. We call this the learner's personalized learning state. For example, an ordinary-differential-equation query may require high-school procedural scaffolding or college-level modeling context. Therefore, tutoring retrieval must find evidence that is pedagogically appropriate for the learner, not merely topical.

Classical RAG can include explicit learner-state text when supplied, but appending learner state, dialogue, grading criteria, and task constraints forces one query embedding to compress topic, level, misconception, instructional goal, and response constraints. As constraints grow, search intent can blur. The issue is therefore how a tutoring system should organize constraints before retrieval and generation.

Therefore, we introduce MARLET (Multi-Agent Retrieval for Learning-State-Aware Educational Tutoring). Instead of letting the retriever act as the top-level controller, a supervisory router identifies the tutoring regime and decides whether retrieval is needed. A planner scopes search intent, a diagnoser summarizes visible learning state, and a rubric module states answer criteria; these modules run in parallel when possible, then are reassembled with retrieved evidence into one tutor prompt. The framework integrates selective retrieval, tool-using agents, and tutoring-state modeling into one context-construction mechanism.

The experiments validate the framework rather than present a model leaderboard. We compare with classical retrieve-then-read RAG and Agentic RAG under shared backbone, corpus, reranker, response cap, tutor, and critic. Our validation on TutorEval and EduBench shows that the same tutor scores higher when fed context assembled by MARLET; on EduBench it also reduces latency and backbone calls.

Contributions. (i) We propose MARLET, a learning-state-aware multi-agent retrieval framework with a supervisory router, planner, diagnoser, rubric module, conditional retriever, tutor, and critic. (ii) We formalize the router and retrieval gate as transparent, training-free controls over learner state, rubric constraints, and external evidence. (iii) We validate against classical RAG and Agentic RAG baselines on two educational benchmarks under shared backbone, corpus, reranker, and response budget.

## II. Related Work

Classical RAG fixes retrieval as the first controller: the query is matched to passages, and generation conditions on the returned evidence [4]. Selective and corrective RAG methods improve this controller. Self-RAG learns

Submitted to the 2026 IEEE International Conference on Systems, Man, and Cybernetics (SMC). Single-blind review.

when to retrieve, generate, and critique; CRAG evaluates retrieved documents and attempts correction when retrieval is unreliable; Adaptive-RAG and TARG vary retrieval behavior by query complexity or retrieval utility [6]–[9]. RAG diagnostics further emphasize separating retrieval, ranking, and generation errors [10]. These methods address whether passages retrieved for a query are necessary, sufficient, or reliable, but they do not infer learner stage before retrieval, separate rubric constraints from topic evidence, or decide whether external passages are the right source of grounding for a tutoring decision.

Agentic RAG and multi-agent LLM systems move beyond a single retrieve-then-read step. ReAct, AutoGen, CAMEL, and MetaGPT show that tool use and role decomposition can improve complex task execution [11]–[14]. MA-RAG uses multiple agents for query disambiguation, evidence extraction, and answer synthesis on ambiguous or multi-hop QA tasks [15]. These systems demonstrate coordination benefits, but their roles are designed for general task execution or ambiguous question answering. They do not define pedagogical regimes, infer visible learner state, or bind the final response to rubric-style answer criteria, so the educational reason for retrieving, skipping retrieval, or prioritizing rubric context remains implicit.

Recent educational work shows that tutoring quality requires more than answer accuracy. AgentTutor builds multi-turn personalized learning with learner assessment and memory [2]; adaptivity studies show that LLM tutors are sensitive to omitted student-context features [16]; and AI-tutor evaluation emphasizes pedagogy, mistake diagnosis, guidance quality, and learner support [1], [17]–[20]. These works define tutoring requirements, but they do not formulate retrieval as learner-state-aware control: when to retrieve, what evidence to retrieve, and how much context to allocate to evidence rather than learner-state and rubric information.

## III. Methodology

MARLET turns one educational request into a grounded tutor response. It is a RAG controller rather than a set of independent answer agents: every intermediate output is converted into context for one final LLM tutor prompt. Its design follows three principles: route before retrieval, exchange typed records rather than free-form chat, and retrieve only when the tutoring brief needs external evidence. The flow is: standardize the entry, score routing cues, run active specialists in parallel, merge their outputs with retrieved evidence, serialize one augmented prompt, and run one critic revision if needed.

### A. Input Entry and Prompt Budget

For one sample, MARLET first standardizes all visible fields into

$$x = (q, o, c, h, r, v, m). \tag{1}$$

TABLE I
Pseudocode for MARLET.

```
def MARLET(x, B, index, thresholds):
  q, o, c, h, r, v, m = x
  scores = cue_scores(concat(q, c, r, h))
  R, switches = route(scores, r, v, thresholds)
  z0 = Context(x=x, regime=R, budget=B)
  tasks = {planner(z0), diagnoser(z0)}
  if switches.rubric: tasks.add(rubric_agent(z0))
  p, ell, u = run_parallel(tasks)
  P = p.queries[:B.K]; z1 = merge(z0, p, ell, u)
  D = []
  if switches.retrieve:
    D = pack(rerank(dedup(search(index, P)), q), B.Q, B.L)
  y0 = tutor(prompt(q, o, h, p, ell, u, D, c))
  if not D and scores.evidence >= thresholds.fallback:
    D = pack(rerank(search(index, [q]), q), B.Q, B.L)
    y0 = tutor(prompt(q, o, h, p, ell, u, D, c))
  y = critic(y0, z1, D) if switches.critic else y0
  return y, {route=R, evidence=D, citations=ids(D)}
```

Here, $q$ is the student question or task prompt; $o$ is an optional answer-option list; $c$ is optional inline context such as a chapter excerpt; $h$ is recent dialogue; $r$ is an optional rubric or answer-criteria list; $v$ is an optional image list; and $m$ stores metadata used for logging or formatting. Empty fields are allowed, so the same representation covers factual questions, grading requests, feedback requests, and planning requests.

The only student-facing output is the final tutor answer $y$. MARLET also stores internal records: a route record, which names the selected tutoring mode and which modules run; an evidence bundle $D$, which contains packed retrieved chunks; citation identifiers, which link chunks in $D$ to their sources; and a cost log for token, latency, and call counts. These records are not answers. Planner output, learner-state summary, rubric summary, and $D$ are assembled as prompt blocks before the LLM tutor generates $y$. The shared budget is

$$B = (K, Q, L, N),$$

where $K$ is the maximum number of planner retrieval queries, $Q$ is the maximum number of evidence chunks/tool calls, $L$ is the character budget for packed evidence, and $N$ is the response-token cap.

### B. Supervisor Routing

The supervisor begins with a routing blob:

$$b(x) = \mathrm{concat}(q, c, r, h), \tag{2}$$

where $\mathrm{concat}(\cdot)$ means that the question, inline context, rubric text, and dialogue turns are joined into one normalized string. Images remain available to the tutor, but they are not converted into the textual routing score.

The router uses five cue families $j \in \{\mathrm{ev}, \mathrm{co}, \mathrm{ru}, \mathrm{pl}, \mathrm{tu}\}$ for evidence, coordination, rubric, planning, and tutoring. Each family controls a different downstream decision: evidence cues open retrieval, coordination cues indicate that several constraints must be merged, rubric cues activate criterion checking, planning cues route to lesson sequencing, and tutoring cues activate learner-state adaptation. Each family has a keyword set $W_j$. Evidence cues include source, document, chapter, and verification; rubric cues include score, criteria, and comment;

planning cues include lesson, sequence, and exercise plan; tutoring cues include student, hint, misconception, explain, and confused. Let $M_j(x)$ be the number of cues from $W_j$ that appear in $b(x)$. The cue score is

$$s_j(x) = \frac{M_j(x)}{|W_j|}, \tag{3}$$

where $|W_j|$ is the number of cues in family $j$, so $s_j(x)$ is the fraction of that cue family observed in the routing blob. The thresholds are fixed router hyperparameters, not probabilities or learned weights. We choose them in cue-count units so $\tau_{\text{mid}}$ requires a repeated cue pattern rather than one accidental word, $\tau_{\text{plan}}$ is stricter because a false planning route changes the whole answer structure, and $\tau_{\text{fb}}$ is stricter because fallback retrieval costs an extra pass.

After cue scoring, the supervisor chooses a pedagogical regime, meaning the prompt-construction policy used before the final LLM call. Lesson planning (LP) organizes a sequence of learning steps; rubric feedback (RF) evaluates an answer under stated criteria; adaptive tutoring (AT) explains, hints, or corrects misconceptions using visible learner state; and evidence-grounded reasoning (EG) answers mainly from source evidence. Because cues can overlap, $R(x)$ is selected by the priority rule

$$R(x) = \begin{cases} \text{LP}, & s_{\text{pl}} \geq s_{\text{ru}},\ s_{\text{pl}} \geq s_{\text{tu}},\ s_{\text{pl}} \geq \tau_{\text{plan}}, \\ \text{RF}, & s_{\text{ru}} \geq s_{\text{tu}},\ s_{\text{ru}} \geq \tau_{\text{mid}}, \\ \text{AT}, & s_{\text{tu}} \geq \tau_{\text{mid}}, \\ \text{EG}, & \text{otherwise}. \end{cases} \tag{4}$$

The priority order is a design choice about prompt constraints. LP is checked first because sequence-level planning reshapes the whole response; RF is next because scoring criteria are hard constraints; AT follows because learner state changes the explanation style but should not override explicit criteria; EG is the fallback so uncertain cases remain grounded rather than answered from memory.

The retrieval gate implements the same selective-retrieval intuition studied in adaptive RAG: evidence is requested when the sample appears to need external grounding rather than being forced for every input [6], [8], [9]. The gate is

$$G(x) = \begin{cases} 1, & s_{\text{ev}} \geq \tau_{\text{mid}} \text{ or } R(x) = \text{EG}, \\ 0, & \text{otherwise}. \end{cases} \tag{5}$$

Thus retrieval runs only when the evidence score reaches $\tau_{\text{mid}}$ or when the selected regime is evidence-grounded. This favors recall for initial grounding. A fallback gate handles missed evidence cases with the stricter $\tau_{\text{fb}}$:

$$G_{\text{fb}}(x) = \begin{cases} 1, & D_0 \text{ is empty and } s_{\text{ev}} \geq \tau_{\text{fb}}, \\ 0, & \text{otherwise}. \end{cases} \tag{6}$$

where $D_0$ is the evidence bundle from the first retrieval attempt. If $G_{\text{fb}}(x) = 1$, the system performs one additional retrieval pass using the raw question.

## C. Parallel Agent Brief Construction

MARLET uses role decomposition for planning, diagnosis, and rubric analysis, following tool-using and multi-agent LLM systems [11], [12]. Planning chooses the local answer strategy and search intents for the current sample; it is not a full curriculum planner. Coordination means typed shared-state assembly: each specialist reads the same input state, writes its own fields, and a coordinator merges them before retrieval and generation. The initial context is

$$z_0 = (x, R(x), G(x), B). \tag{7}$$

From $z_0$, the planner, diagnoser, and rubric module run in parallel when their switches are active:

$$(p, \ell, u) = \big(A_{\text{plan}}(z_0), A_{\text{diag}}(z_0), A_{\text{rub}}(z_0)\big). \tag{8}$$

Here $p$ is the planner output, consisting of a response strategy and up to $K$ retrieval queries; $\ell$ is the visible learner-state summary; and $u$ is the rubric summary or default checklist. In parallel execution, all active agents read $z_0$ and return typed outputs independently, so the tutor need not infer plan, learner state, and rubric from one overloaded query.

The planner starts its query set with $q$, adds a compact key-term query from longer tokens in $q$, adds a context-aware query when $c$ is present, and adds a rubric-aware query when $r$ is present. Duplicate queries are removed and the list is capped by $K$. The diagnoser reflects tutoring work showing that LLM tutor quality depends on student-context features [2], [16]. It infers only visible state from $q$ and $h$: level defaults to intermediate, beginner or advanced cues can override it, dialogue cues can add misconceptions, and style cues can request step-by-step, concise, or analogy-based explanation. This is not a persistent student profile. The rubric module preserves explicit criteria when available; otherwise it supplies default criteria of correctness, clarity, scaffolding, and actionable feedback. The merged tutoring brief is

$$z_1 = (z_0, p, \ell, u). \tag{9}$$

## D. Conditional Retrieval and Evidence Packing

MARLET contains retrieval, but retrieval is not the top-level controller and does not require a traditional dense vector database. When $G(x) = 1$, each planner query $q_i$ searches a local hybrid text index. For a query $q_i$ and candidate chunk $d$, the base retrieval score is

$$\text{base}(q_i, d) = w_{\text{lex}}\text{lex} + w_{\text{char}}\text{char} + w_{\text{svd}}\text{svd} + \beta. \tag{10}$$

Here lex is word TF–IDF similarity, char is character 3–5 gram TF–IDF similarity, svd is truncated-SVD similarity, $\beta$ is a small title/source boost, and the $w$ terms are fixed retriever weights. Candidates from all planner queries are merged, and duplicate chunks keep their highest score.

The reranker then scores each candidate against the original question:

$$\begin{aligned} \text{rank}(q, d) = &\alpha_{\text{base}}\text{base} + \alpha_{\text{tok}}o_{\text{tok}} + \alpha_{\text{title}}o_{\text{title}} \\ &+ \alpha_{\text{phr}}m_{\text{phr}} + \alpha_{\text{num}}o_{\text{num}} + \alpha_{\text{len}}b_{\text{len}}. \end{aligned} \tag{11}$$

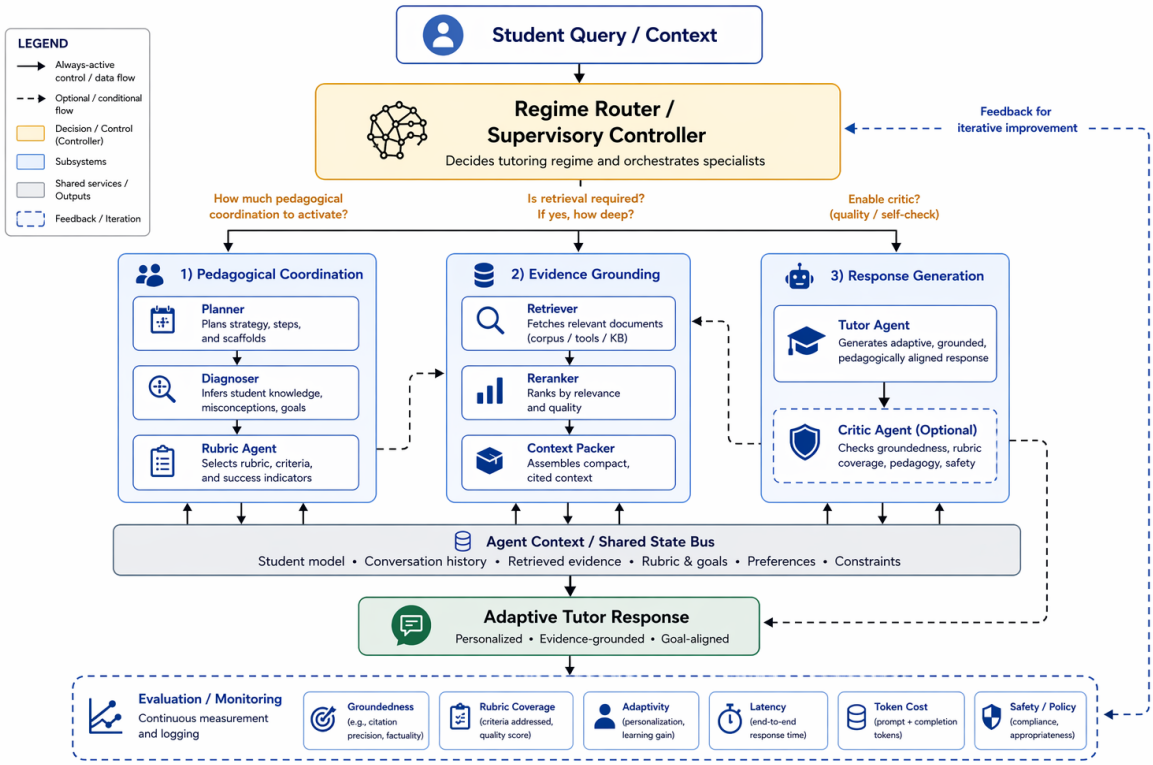


Fig. 1. Main framework of MARLET. The router scores the entry, activates specialist modules, and decides whether retrieval is needed; specialist outputs and retrieved evidence are then assembled into one augmented tutor prompt.

Here $o_{\text{tok}}$ is token overlap with $q$, $o_{\text{title}}$ is title-token overlap, $m_{\text{phr}}$ is an exact phrase-match flag, $o_{\text{num}}$ is numeric overlap, $b_{\text{len}}$ rewards moderate chunk length, and the $\alpha$ terms are fixed reranker weights. The learner summary $\ell$ affects retrieval through the route and planner queries, not these weights.

Evidence packing follows the relevance-diversity tradeoff used in MMR-style selection [21]: the context should be useful, but repeated chunks should not consume the limited evidence budget. For each candidate chunk $d$, MARLET computes

$$\text{pack}(d) = \lambda \,\text{rank}(q, d) - (1 - \lambda)\,\text{repeat}(d). \tag{12}$$

Here $\text{repeat}(d)$ is the largest sentence overlap between $d$ and selected chunks, and $\lambda$ controls the relevance-diversity balance. The packer adds the remaining chunk with the highest $\text{pack}(d)$ score until $Q$ chunks or the $L$ character limit is reached, forming evidence bundle $D$.

## E. Tutor Generation and Critic Revision

The final tutor context is

$$z_2 = (z_1, D). \tag{13}$$

This is where retrieval and agent outputs converge: $z_1$ supplies route, plan, learner-state, and rubric fields, while $D$ supplies compact cited evidence. The retriever does not answer; its chunks become an evidence block. The final prompt contains task fields, pedagogical brief, evidence or inline context, and output requirements. The tutor is the LLM-driven answer writer:

$$y_0 = A_{\text{tutor}}(z_2). \tag{14}$$

Its prompt contains $q$, optional answer options $o$, recent dialogue $h$, learner state $\ell$, plan $p$, rubric summary $u$, and either retrieved evidence $D$ or inline context $c$. The tutor is instructed to be correct, pedagogically useful, concise but not terse, include a next step when appropriate, and cite retrieved evidence with document markers.

The critic performs one bounded revision pass:

$$y = \begin{cases} A_{\text{crit}}(y_0, z_2), & I(y_0, z_2) > 0, \\ y_0, & I(y_0, z_2) = 0. \end{cases} \tag{15}$$

Here $I(y_0, z_2)$ is the number of issues found by the critic. Issues include missing citation markers when $D$ is nonempty, uncovered rubric items from $r$, and answers that are too long for a beginner-facing state. If no issue is found, $y_0$ is returned unchanged; otherwise the critic rewrites once and the revised $y$ becomes the final answer.

# IV. Validation Experiments

We validate the framework on two public educational benchmarks. TutorEval [1] contains 834 expert-written science tutoring questions paired with long STEM chapters and teaching key points, so responses must explain, disambiguate, and stay chapter-grounded. EduBench [18] covers nine educational scenarios, including question answering, error correction, personalized support, emotional support, grading, teaching-material generation, and personalized content creation. Its rows

contain prompts, scenario metadata, and reference model predictions; our adapter converts them into consensus-style supervision through score statistics and rubric terms.

The primary validation compares three context controllers under matched infrastructure, with a Qwen3.5-27B-FP8 replication checking backbone sensitivity. Classical RAG retrieves first from the raw standardized prompt; visible learner profile, student answer, dialogue, and rubric text remain available to its tutor prompt, so the baseline is not deprived of pedagogical context. Its limitation is controller order: retrieval happens before specialist summaries can shape search. Agentic RAG runs the same planner, diagnoser, and rubric modules before retrieval, but retrieval is always on. MARLET keeps the same specialists and retriever, then adds supervisory routing, conditional retrieval, and fallback control. We report quality and cost together because only context construction changes.

## A. Implementation Details

All controllers use the same deployed Python framework, corpus, hybrid text index, reranker, tutor, critic, and metric code; only context assembly differs. Runs use Qwen3.5-4B and Qwen3.5-27B-FP8, reasoning budget 512, temperature 0.15, and response cap $N$=900. The checked server benwulab-Lambda-Vector runs Ubuntu Linux 6.8 and Python 3.10.12 with two NVIDIA RTX 6000 Ada GPUs (49140 MiB each), driver 570.207, and CUDA 12.8. We set $K$=3, $Q$=4, and $L$=6000; router thresholds $(\tau_{\text{mid}}, \tau_{\text{plan}}, \tau_{\text{fb}}) = (0.35, 0.42, 0.45)$; retriever weights $(w_{\text{lex}}, w_{\text{char}}, w_{\text{svd}}) = (0.58, 0.17, 0.25)$; reranker weights $(\alpha_{\text{base}}, \alpha_{\text{tok}}, \alpha_{\text{title}}, \alpha_{\text{phr}}, \alpha_{\text{num}}, \alpha_{\text{len}}) = (0.55, 0.20, 0.10, 0.06, 0.05, 0.04)$; and packing weight $\lambda$=0.68.

Evaluation is dataset-aware, and all reported quality scores are automatic proxies rather than official benchmark or human grades. Higher is better for quality metrics; lower is better for latency, tokens, and calls. Following RAG evaluation work [10], we separate answer overlap, rubric fulfillment, grounding, and resource cost. Token-F1 rewards overlap with the reference while balancing extra generated tokens against missed reference tokens. Rubric coverage checks whether each benchmark rubric item is actually expressed; near-verbatim coverage receives credit, and partial token overlap must include content-bearing words so that generic feedback is not over-rewarded. Latency is wall-clock time per sample; total tokens are prompt plus completion tokens; backbone calls count logged LLM requests.

For EduBench, EB12D averages 12 normalized checks in three groups: scenario adaptation, factual or reasoning behavior, and pedagogical application. These checks cover output-format compliance, supportive tone, relevance, use of scenario details, score agreement, domain and rubric grounding, reasoning quality, correction cues, clarity, motivation, personalization, and higher-order prompting. EduAlign is narrower: it only measures how close the extracted numeric score is to the benchmark's consensus score. Thus EduAlign can decrease even when explanatory feedback or rubric coverage improves. For TutorEval, tutor-hit measures expert teaching-point coverage. A key point receives full credit when it is explicitly stated or strongly covered, partial credit when the answer overlaps with enough of the key point, and no credit when the idea is absent. Off-profile metrics remain zeros in raw logs but are omitted from paper tables. The workload audit is a fixed weighted cost summary over tokens, retrieval queries, retrieved chunks, materialized agent outputs, and trace events; it is for efficiency inspection only and is not a training objective.

TABLE II

4B quality and resource summary.

| Data | Metric | RAG | Ours | Δ |
|---|---|---|---|---|
| TutorEval | Token-F1 | 0.112 | 0.115 | $+0.004^{\star}$ |
| | TE hit | 0.938 | 0.957 | $+0.019^{\star\star}$ |
| | Rubric | 0.919 | 0.944 | $+0.026^{\star\star}$ |
| | Latency (s) | 51.90 | 49.11 | $-2.79$ |
| | Tokens | 3975 | 3490 | $-484^{\star\star\star}$ |
| | Calls | 1.84 | 1.56 | $-0.28$ |
| EduBench | EB12D | 0.367 | 0.398 | $+0.031^{\star\star\star}$ |
| | Rubric | 0.705 | 0.891 | $+0.186^{\star\star\star}$ |
| | EduAlign | 0.166 | 0.126 | $-0.040^{\star\star}$ |
| | Latency (s) | 19.83 | 13.90 | $-5.92^{\star\star\star}$ |
| | Tokens | 4020 | 2226 | $-1794^{\star\star\star}$ |
| | Calls | 1.97 | 1.48 | $-0.49^{\star\star\star}$ |

$^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency, tokens, and calls. Runtime is mean wall-clock seconds/sample on the experiment server.

TABLE III

27B quality and resource summary.

| Data | Metric | RAG | Ours | Δ |
|---|---|---|---|---|
| TutorEval | Token-F1 | 0.264 | 0.272 | $+0.008^{\star}$ |
| | TE hit | 0.897 | 0.918 | $+0.021^{\star}$ |
| | Rubric | 0.861 | 0.891 | $+0.030^{\star}$ |
| | Latency (s) | 41.47 | 40.65 | $-0.82^{\star\star}$ |
| | Tokens | 6681 | 6730 | $+49$ |
| | Calls | 1.99 | 1.99 | $-0.01$ |
| EduBench | EB12D | 0.355 | 0.425 | $+0.070^{\star\star\star}$ |
| | Rubric | 0.634 | 0.769 | $+0.135^{\star\star\star}$ |
| | EduAlign | 0.541 | 0.526 | $-0.015$ |
| | Latency (s) | 41.91 | 32.09 | $-9.81^{\star\star\star}$ |
| | Tokens | 6105 | 3717 | $-2388^{\star\star\star}$ |
| | Calls | 2.00 | 1.81 | $-0.19^{\star\star\star}$ |

TutorEval uses $n$=400 paired samples; EduBench uses $n$=328 unique paired rows after duplicate public IDs in the 400-example slice are collapsed. $^{\star\star\star}p<10^{-4}$, $^{\star\star}p<0.01$, $^{\star}p<0.05$. Higher is better for quality; lower is better for latency, tokens, and calls. Runtime is mean wall-clock seconds/sample on the experiment server.

## B. Framework Validation

Table II reports paired 4B comparisons with bootstrap CIs and permutation p-values. Agentic RAG uses the same specialist brief as MARLET but retrieves for every sample; its matched results are being generated under the same protocol. On TutorEval, MARLET raises Token-F1, tutor-hit, and rubric coverage while using fewer tokens and fewer calls; latency trends lower but is not reliable ($p$=0.21). Table III reports the 27B replication.

### C. Trade-offs

For TutorEval, the 27B paired CIs are positive for Token-F1, tutor-hit, rubric coverage, and latency. For EduBench, EB12D and rubric coverage improve, latency/tokens/calls decrease, and the small classical-RAG EduAlign edge is not statistically reliable.

MARLET can use more modules than classical RAG, so resource cost must be reported. Scoped planner queries and the retrieval gate reduce token use in 4B and still reduce latency in 27B, but 27B TutorEval shows that better coverage does not always mean fewer tokens. Thus MARLET is a controller for deciding what grounding a tutoring sample needs, not a universal retrieval replacement.

## V. Limitations

The study validates a framework rather than a deployable tutor: evaluation is automatic and English-only, and learner state is inferred from visible prompt/dialogue fields. Deployment requires privacy safeguards, bias checks, data governance, human oversight, and learning-gain studies.

## VI. Conclusion

MARLET reframes educational grounding as supervised context construction rather than query-only passage lookup. With shared infrastructure, it improves fixed-tutor quality and efficiency over classical RAG by coordinating learner state, rubric constraints, and conditional evidence before generation. Future work should test stronger controller baselines, human and multi-turn outcomes, learned router calibration, profile-aware reranking, and privacy-preserving learner-state inference.

## References


[1] A. Chevalier, J. Geng, A. Wettig, H. Chen, S. Mizera, T. Annala, M. J. Aragon, A. R. Fanlo, S. Frieder, S. Machado, A. Prabhakar, E. Thieu, J. T. Wang, Z. Wang, X. Wu, M. Xia, W. Xia, J. Yu, J.-J. Zhu, Z. J. Ren, S. Arora, and D. Chen, "Language models as science tutors," arXiv:2402.11111, 2024. [Online]. Available: https://arxiv.org/abs/2402.11111

[2] Y. Liu, Z. Song, J. Lou, C. Wu, and J. Li, "AgentTutor: Empowering personalized learning with multi-turn interactive teaching in intelligent education systems," arXiv:2601.04219, 2026. [Online]. Available: https://arxiv.org/abs/2601.04219

[3] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, and J. Weston, "Wizard of Wikipedia: Knowledge-powered conversational agents," in Proceedings of the International Conference on Learning Representations, 2019. [Online]. Available: https://arxiv.org/abs/1811.01241

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[5] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2021. [Online]. Available: https://openreview.net/forum?id=wCu6T5xFjeJ

[6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in International Conference on Learning Representations, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv:2401.15884, 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 7036–7050. [Online]. Available: https://aclanthology.org/2024.naacl-long.389/

[9] Y. Wang, L. Wei, and H. Ling, "Retrieval as a decision: Training-free adaptive gating for efficient RAG," arXiv:2511.09803, 2026. [Online]. Available: https://arxiv.org/abs/2511.09803

[10] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2408.08067

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in International Conference on Learning Representations, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[12] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2024. [Online]. Available: https://arxiv.org/abs/2308.08155

[13] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," 2023. [Online]. Available: https://arxiv.org/abs/2303.17760

[14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[15] T. Nguyen, P. Chin, and Y.-W. Tai, "MA-RAG: Multi-agent retrieval-augmented generation via collaborative chain-of-thought reasoning," arXiv:2505.20096, 2025. [Online]. Available: https://arxiv.org/abs/2505.20096

[16] C. Borchers and T. Shou, "Can large language models match tutoring system adaptivity? A benchmarking study," in Artificial Intelligence in Education. Springer Nature Switzerland, 2025, pp. 407–420. [Online]. Available: https://arxiv.org/abs/2504.05570

[17] K. K. Maurya, K. A. Srivatsa, K. Petukhova, and E. Kochmar, "Unifying AI tutor evaluation: An evaluation taxonomy for pedagogical ability assessment of LLM-powered AI tutors," in Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025. [Online]. Available: https://aclanthology.org/2025.naacl-long.57/

[18] B. Xu, Y. Bai, H. Sun, Y. Lin, S. Liu, X. Liang, Y. Li, Z. Dong, J. Zhang, Y. Deng, X. Zou, Y. Gao, and H. Huang, "EduBench: A comprehensive benchmarking dataset for evaluating large language models in diverse educational scenarios," arXiv:2505.16160, 2025. [Online]. Available: https://arxiv.org/abs/2505.16160

[19] J. Macina, N. Daheim, I. Hakimi, M. Kapur, I. Gurevych, and M. Sachan, "MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors," in Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 204–221. [Online]. Available: https://aclanthology.org/2025.emnlp-main.11/

[20] R. S. Srinivasa, Z. Che, C. B. C. Zhang, D. Mares, E. Hernandez, J. Park, D. Lee, G. Mangialardi, C. Ng, E.-Y. H. Cardona, A. Gunjal, Y. He, B. Liu, and C. Xing, "TutorBench: A benchmark to assess tutoring capabilities of large language models," arXiv:2510.02663, 2025. [Online]. Available: https://arxiv.org/abs/2510.02663

[21] J. Carbonell and J. Goldstein, "The use of mmr, diversity-based reranking for reordering documents and producing summaries," in Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 1998, pp. 335–336.